

Objectifs

Dans ce travail pratique vous devez mettre en œuvre les deux techniques fondamentales d'exploration d'un graphe, l'exploration en largeur et celle en profondeur, et les appliquer à la création et à la résolution de labyrinthes.

Création de labyrinthes

Il existe de nombreuses approches pour générer aléatoirement des labyrinthes. L'une des plus simples repose sur l'exploration en profondeur d'un graphe. Plus spécifiquement on considère une grille de n rangées par m colonnes à laquelle on associe un graphe dont les sommets correspondent aux cellules de la grille et où deux sommets sont reliés par une arête si les cellules associées sont contigües (horizontalement ou verticalement). La situation est illustrée en figure 1, à gauche, pour une grille de 4 rangées par 5 colonnes.

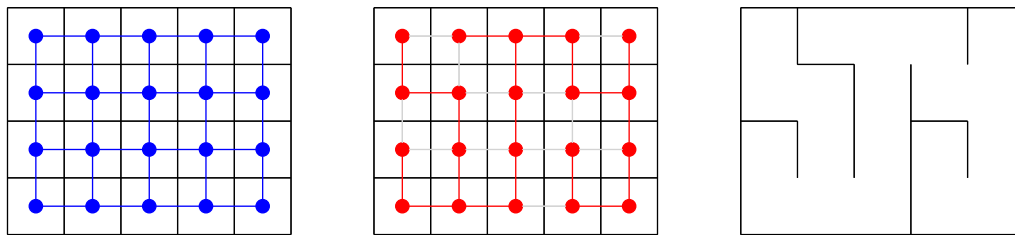


FIGURE 1 – À gauche, la grille de départ de 4 rangées par 5 colonnes avec, en bleu, le graphe associé. Au centre, un arbre recouvrant, en rouge, du graphe associé à la grille. À droite, le labyrinthe obtenu à partir de l'arbre recouvrant. Chaque arête de l'arbre correspond à un déplacement possible entre les cellules associées à ses extrémités.

On choisit ensuite un sommet du graphe comme point de départ d'une exploration en profondeur dans laquelle les voisins de chaque sommet sont traités dans un ordre aléatoire. On obtient alors un arbre recouvrant comme illustré en figure 1 au centre. Pour chaque arête de cet arbre on supprime le segment correspondant de la grille, autorisant ainsi le passage entre les deux cellules associées aux extrémités de l'arête. Comme résultat on obtient un labyrinthe, généré aléatoirement, comme illustré en figure 1 à droite.

Dans ce travail pratique, vous devez mettre en œuvre l'approche décrite ci-dessus pour générer un labyrinthe aléatoire en complétant, dans les sources fournies, l'implémentation de la

classe `DfsGenerator` (implémentant l'interface `MazeGenerator`). Vous veillerez à respecter les deux points suivants :

- ▷ programmer une exploration en profondeur **non récursive** pour le calcul de l'arbre recouvrant ;
- ▷ pour chaque sommet, au début de son traitement, récupérer la liste de ses voisins et la mélanger aléatoirement **une seule fois** à l'aide de la méthode `shuffle` de la classe `ArrayUtil` fournie.

Résolution de labyrinthes

Les labyrinthes obtenus à partir d'une exploration en profondeur ne sont pas exempts de défauts. En particulier ils ne possèdent qu'un seul itinéraire permettant de rejoindre la sortie, quels que soient le point de départ et la sortie.

Afin d'améliorer un peu les choses, l'interface graphique fournie permet de détruire aléatoirement une petite proportion des murs du labyrinthe afin d'offrir de nouveaux déplacements possibles.

Maintenant, pour résoudre un labyrinthe, une exploration en largeur permet de déterminer un itinéraire entre le point de départ et la sortie. Une telle exploration a l'avantage de déterminer un itinéraire le plus court possible lorsque plusieurs solutions existent.

Dans ce travail pratique, vous devez mettre en œuvre la résolution d'un labyrinthe à l'aide d'une exploration en largeur en complétant l'implémentation de la classe `BfsSolver` (implémentant l'interface `MazeSolver`). Votre algorithme devra calculer un itinéraire de sortie optimal (minimisant le nombre de déplacements), sa longueur ainsi que le nombre de solutions optimales. L'algorithme de la table 1 propose un pseudocode permettant de calculer le nombre de plus courtes chaînes entre un sommet source s et les autres sommets d'un graphe non orienté.

Modalités et délais

- ▷ Le travail de programmation est à effectuer par groupe de deux, en Java, version 25.
- ▷ L'archive contenant les sources du projet est disponible sur le site Cyberlearn du cours.
- ▷ Vous devez rendre une archive (au format `zip`) contenant le répertoire `GroupX` (au nom ajusté selon votre groupe) et uniquement ce répertoire.
- ▷ Vous devez rendre votre travail sur Cyberlearn au plus tard le **dimanche 29 mars 2026** (avant minuit).

TABLE 1 – Exploration en largeur d'un graphe G à partir d'un sommet source s .

Algorithme BFS(G, s)

Données : Un graphe non orienté $G = (V, E)$ connexe et un sommet source $s \in V$.

Résultat : Pour chaque sommet $u \in V$, la distance $d[u]$ séparant s et u (nombre d'arêtes d'une plus courte chaîne entre s et u), le « père » $p[u]$ de u (codant l'arbre d'exploration en largeur) et le nombre $N[u]$ de plus courtes chaînes entre s et u .

Début

- (1) **Pour chaque** sommet $u \in V \setminus \{s\}$ **poser**
- (2) $d[u] \leftarrow -1$, $p[u] \leftarrow \text{null}$ et $N[u] \leftarrow 0$
- (3) **Poser** $d[s] \leftarrow 0$, $p[s] \leftarrow \text{null}$ et $N[s] \leftarrow 1$
- (4) Créer une file (FIFO) Q et y introduire s
- (5) **Tant que** Q n'est pas vide **faire**
- (6) Retirer le premier sommet u de Q
- (7) **Pour chaque** sommet voisin v de u **faire**
- (8) **Si** $d[v] = -1$ **faire** // découverte de v
- (9) **Poser** $d[v] \leftarrow d[u] + 1$, $p[v] \leftarrow u$ et $N[v] \leftarrow N[u]$
- (10) Insérer le sommet v à la fin de Q
- (11) **Sinon si** $d[v] = d[u] + 1$ **faire** // itinéraire alternatif
- (12) **Poser** $N[v] \leftarrow N[v] + N[u]$
- (13) Retourner les tableaux d , p et N

Fin